Security Agent Runtime

重要文件与关键函数源码学习讲义

v0.1 至 v0.7 七轮迭代
安全研发 / 安全工程面试复习

基于 E:/security_agent 当前源码
提交 edd818c · 2026-11-05
配套 16 页面试展示 PPT，含逐页讲稿。

阅读目标：理解调用链、解释安全边界、定位实现函数，并能用测试验证自己的理解。

目录

01 阅读路线与材料边界 / 03

02 七轮版本演进 / 04

03 重要文件导航 / 05

04 五种数据与两类上下文 / 06

05 入口与依赖注入 / 07

06 主循环 AgentRuntime / 08

07 结构化提案与语义校验 / 09

08 任务编译与 ID 所有权 / 10

09 规划上下文的白名单 / 11

10 DemoPlanner 与 LLMPlanner / 12

11 依赖图校验 / 13

12 就绪刷新与失败传播 / 14

13 Supervisor.delegate 执行入口 / 15

14 Workflow、Policy、Scope 与 Capability / 16

15 审批快照与状态存储 / 17

16 Supervisor.approve 批准后复检 / 18

17 Supervisor._execute 与结果校验 / 19

18 StateManager 与 Finding 关联 / 20

19 EvidenceExtractor 与 FindingEvaluator / 21

20 工具契约与注册表 / 22

21 ToolExecutor.execute 工具权限 / 23

22 CodeAnalysisAgent 与报告 Agent / 24

23 文件路径校验与有界读取 / 25

24 ReadSourceTool.execute / 26

25 SearchCodeTool.execute / 27

26 HTTP 参数与 Origin 限制 / 28

27 真实 HTTP 请求与 baseline 观察 / 29

28 端到端源码搜索推演 / 30

29 安全测试的阅读方法 / 31

30 复习实验与运行命令 / 32

31 面试高频问题与回答要点 / 33

32 局限、改进与个人贡献表达 / 34

01 阅读路线与材料边界

这份讲义围绕当前 v0.7 代码解释 v0.1 至 v0.7 的演进。先掌握执行主链，再沿关键函数回到源码。

材料基线

项目目录 E:/security_agent，源码提交 edd818c。整理日期 2026-11-05。版本映射来自 Git 提交、已有 tag、CHANGELOG 与 README，不把历史会话里的旧测试数量当作当前结果。函数行号以此快照为准，后续改代码可能变化。

建议分三次学习

第一次：阅读版本演进、数据模型和端到端调用，能用 90 秒介绍项目。第二次：重点看 Supervisor、审批快照、Scheduler 和工具权限，手动推演失败分支。第三次：运行指定测试并回答每章自测，再练习 PPT 讲稿。

真实实现与教学实现

真实实现包括源码搜索、读取、受控 HTTP 基线与执行控制。verify_sqli、verify_ssrf 和 ReportAgent 仍是教学模拟。当前没有通用漏洞扫描、真实利用链、持久化恢复、并发调度或恶意插件沙箱。

验证事实

本次运行 unittest discover -s tests -q，181 项通过。另运行 main.py --target ./tools，扫描 6 个文件，字面命中 4 处，2 个任务完成，Finding/Evidence 均为空。没有调用真实外部模型 API。

02 七轮版本演进

按“这一轮补上什么控制能力”记忆，比背文件名更容易在面试中说明设计理由。

v0.1 / v0.2

757b614：建立 AgentRegistry、Supervisor 与最小派发链。92db104：加入请求校验及 Workflow、Capability、Policy、Scope 门控，避免任意任务直接进入 Agent。

v0.3 / v0.4

39e82fd：加入 PendingAction 和审批快照，批准后重新验证。8f27aaa：区分 Observation、Evidence 与 Finding，通过 StateManager 和规则更新漏洞状态。

v0.5 / v0.6

9fbba3c：引入依赖图、READY 调度、失败传播与恢复执行。9da3a73：加入结构化计划、语义校验、任务编译和 Phase 级重新规划。

v0.7

a1406b1 至 edd818c：新增任务 arguments、工具模型、ToolRegistry、ToolExecutor、真实源码与 HTTP 工具，并接通 Agent。最后修正 baseline 元数据存储，增加真实回环请求测试。

面试讲法

前五轮建立确定性的执行底座，第六轮让 LLM 提出计划，第七轮让受控工具产生真实 I/O。七轮迭代不是七个独立产品；不是每一轮都有同名 Git tag。历史能力以当前实现为讲解落点。

03 重要文件导航

学习顺序建议从入口逐层深入，先不要直接从最长的 Supervisor 文件开始。

入口与数据

main.py 负责装配、命令行和输出；state.py 保存当前运行状态；models/task.py 是内部可变任务，models/plan.py 是模型提案，models/approval.py 是不可变快照，models/tool.py 是工具上下文及结果契约。

计划与调度

planner/demo_planner.py 与 planner/llm_planner.py 提供相同的 plan_phase 接口。runtime/orchestrator.py 驱动阶段，plan_validator.py 验证提案，plan_compiler.py 分配 ID，planning_context.py 限制回传信息，scheduler.py 处理依赖。

执行与结果

runtime/supervisor.py 是任务执行边界；approval.py 管审批状态；workflow.py、policy.py、scope.py 分别管阶段、策略、目标。state_manager.py、evidence.py、evaluator.py、finding_rules.py 维护结果及关系约束。

Agent 与工具

agents/ 只执行动作并返回观察。runtime/tool_executor.py 管工具权限，tool_registry.py 管工具注册。tools/file_policy.py 提供路径与资源限制，search_code.py、read_source.py、controlled_http.py 实现实际 I/O。

测试与旧接口

tests/ 包含确定性和回环集成测试；integration_tests/ 的真实模型测试需主动启用。runtime/runner.py 是早期已存在任务的执行循环；当前 main.py 主链使用 AgentRuntime，不调用 RuntimeRunner。

04 五种数据与两类上下文

执行状态、观察信号和漏洞结论分属不同对象，混用会造成“任务成功就是漏洞成立”的错误。

Task 与 PlanProposal

PlannedTask.key 是模型提案里的局部名，depends_on 引用同一提案的 key。编译后 Task.id 由 Runtime 生成，dependencies 保存 Runtime ID。Task.arguments 保存业务参数；target 表示授权源码目标字符串。

Observation / Evidence / Finding

Observation 记录 agent、task_id、execution_status 和 data。Evidence 用 finding_id、task_id 连接任务与发现，并带 WEAK/MEDIUM/STRONG。Finding 保存类型、目标与 POTENTIAL、UNCONFIRMED、CONFIRMED、REJECTED。

ApprovalSnapshot

冻结待执行请求，arguments_json 是规范化 JSON 字符串，dependencies 是 tuple。arguments 属性每次反序列化出新字典，避免共享可变参数。

ToolContext 与 PlanningContext

ToolContext 由 Supervisor 从 State 生成，携带可信的授权根目录和 Origin。PlanningContext 是给 Planner 的有限信息摘要，不是可变 State，也不提供工具执行权限。

自测

把 HTTP 200 写进 Observation 是否能生成 Evidence？不能。EvidenceExtractor 只处理特定验证动作且要求 finding_id，baseline 数据也独立放在 http_baseline 中。

函数定位（项目内相对路径与当前行号）

models/task.py : 6-26 Task

models/plan.py : 14-32 PlannedTask

models/observation.py : 6-22 Observation

models/tool.py : 26-41 ToolContext

05 入口与依赖注入

main.py 将工具和 Agent 装配到同一个 Runtime，便于测试替换 Planner，而不用改变执行安全链。

build_tool_executor()

创建 ToolRegistry 并注册 ReadSourceTool、SearchCodeTool 和 ControlledHttpCheckTool。随后构造（agent, action）到工具名集合的白名单。不存在的组合默认没有工具权限。

build_registry(tool_executor)

把同一个 ToolExecutor 注入 CodeAnalysisAgent 和 VerificationAgent，并分别登记 capability。ReportAgent 只提供 generate_report 演示动作。注册 capability 不会自动给工具权限。

build_runtime(planner)

统一创建 StateManager、WorkflowEngine、Supervisor、TaskScheduler、PlanValidator、PlanCompiler、PlanningContextBuilder 和 PhaseManager，传给 AgentRuntime。依赖注入使 Mock Planner 或测试 Agent 更容易接入。

main() 与 cli()

main 新建 AgentState，运行并打印权威 Runtime 状态。cli 解析 --target、--origin、--goal、--model；没有 --model 使用 DemoPlanner。退出码为 0 的条件是工作流 COMPLETED 且 failed_task_ids 为空。默认 demo_app 不存在，应明确传入现有目录。

自测

把 --origin 加到默认演示会自动请求 HTTP 吗？不会，它只设置授权范围，DemoPlanner 没有生成 baseline 任务。

函数定位（项目内相对路径与当前行号）

main.py : 28-37 build_tool_executor

main.py : 40-46 build_registry

main.py : 49-60 build_runtime

main.py : 88-99 cli

06 主循环 AgentRuntime

run_until_blocked(goal, state) 把规划、调度、人工暂停和阶段推进连接起来。

输入和返回

输入为用户目标与同一运行中的 AgentState。返回 BLOCKED 表示无法继续，WAITING_CONFIRMATION 表示等待 action_ids，COMPLETED 表示阶段到达 END，并附 failed_task_ids。

规划过程

首先校验现有任务图。若当前 phase 不在 planned_phases，构建上下文并调用 Planner，随后验证提案、编译任务。使用临时 SimpleNamespace 校验“现有任务 + 新任务”的完整图，成功后才加入任务并标记该阶段已规划。

执行过程

_run_ready_tasks() 每次刷新就绪状态，以串行循环调用 Supervisor.delegate。记录调用前后的状态，若 READY 任务完全没有进展则返回失败，避免无限空转。

暂停和推进

若当前阶段存在 WAITING_CONFIRMATION，查找绑定此 State 的待批 action_id 并返回。没有等待时由 PhaseManager 判断是否可推进。再次调用时不会重新规划已经标记的阶段。

失败分支

PlannerError、PlanValidationError、TaskGraphError 转为 BLOCKED，规划失败不写入阶段标记，可在修正后重试。这是先验证后提交的顺序保证，不是数据库事务。自测：空 tasks 为什么能推进？该阶段已标记规划且不存在未完成任务。

函数定位（项目内相对路径与当前行号）

runtime/orchestrator.py : 23-81 AgentRuntime.run_until_blocked

runtime/orchestrator.py : 83-93 AgentRuntime._run_ready_tasks

07 结构化提案与语义校验

Pydantic 负责数据形状，PlanValidator 负责数据在当前运行中的含义。

PlanProposal / PlannedTask

提案只包含 phase、summary 和 tasks，模型不能直接提交 Runtime ID。任务字段包括 agent、action、target、arguments、depends_on、finding_id。extra="forbid" 拒绝模型添加未定义的字段。

validate() 的主要顺序

先对 model_dump() 重新做 model_validate，避免用 model_construct 或事后修改绕过校验。然后核对 expected_phase、state.phase 和任务数上限 10，检查 key 唯一性。逐任务检查 Agent 存在、capability、阶段动作和目标精确相等。

Finding 与依赖

finding_error(..., require_finding=True) 要求 SQLi/SSRF 验证引用已有且匹配的未终结 Finding。依赖必须引用当前提案中的 key，不允许自依赖。_check_cycles 使用 DFS 的 visiting 与 visited 集合检测环。

错误示例

不存在的 Agent 得到 UNKNOWN_AGENT；target 改为其他目录得到 TARGET_OUT_OF_SCOPE；A 依赖 B、B 依赖 A 得到 CYCLIC_PLAN。错误会阻止任务写入 State，而不是部分接受合法任务。

自测

已通过 PlanValidator 为何还要 Supervisor？计划之后状态可能变化，且手动派发接口可以不经过 Planner，所以执行必须独立检查。

函数定位（项目内相对路径与当前行号）

models/plan.py : 35-43 PlanProposal

runtime/plan_validator.py : 22-134 PlanValidator.validate

runtime/plan_validator.py : 136-177 PlanValidator._check_cycles

08 任务编译与 ID 所有权

PlanCompiler.compile(proposal, state) 将已校验的局部计划转换成 Runtime Task 列表，不直接修改 State。

两轮编译

第一轮收集 existing_ids，从 T1 开始寻找未占用 ID，并建立局部 key 到 ID 的 mapping。第二轮构造 Task，将 depends_on 的每个 key 映射为 dependencies 中的真实 ID。

复制参数

arguments 使用 deepcopy，避免后续修改提案参数时连带改变 Runtime Task。phase 来自提案，finding_id 保留引用，Task 默认状态为 PENDING。

具体例子

已有 T1，提案包含 key=search 和 key=read，read.depends_on=["search"]。编译结果是 search 对应 T2，read 对应 T3，T3.dependencies=["T2"]。提案本身不能指定 T2 来覆盖现有任务。

与提交的关系

Compiler 返回候选列表后，AgentRuntime 仍要检查整个任务图，才调用 StateManager.add_task。这样既防止 ID 冲突，也避免错误依赖图污染当前运行。

自测

如果直接调用 compile 传入未知 key 会怎样？它不是完整验证入口，映射时可能抛错。调用契约要求先 validate，再 compile，最后验证完整图。

源码节选（省略空行，仅展示函数开头）

```
 9      def compile(self, proposal, state):
10          existing_ids = {task.id for task in state.tasks}
11          mapping = {}
12          next_number = 1
13          for planned in proposal.tasks:
14              while f"T{next_number}" in existing_ids:
15                  next_number += 1
16              task_id = f"T{next_number}"
17              mapping[planned.key] = task_id
18              existing_ids.add(task_id)
19              next_number += 1
21          return [
22              Task(
```

函数定位（项目内相对路径与当前行号）

runtime/plan_compiler.py : 9-33 PlanCompiler.compile

09 规划上下文的白名单

PlanningContextBuilder 只给 Planner 必要的有限信号，避免把任意工具输出当成下一次规划指令。

build() 返回什么

授权目标、当前阶段允许动作、Agent capability、受限 Finding 列表、最近观察以及已完成任务 ID。最多 50 个 Finding、5 条观察、30 个已完成任务 ID，并给出 findings_omitted。

_observations() 怎样筛选

仅保留 boolean_difference、callback_received、request_completed 且值类型严格为 bool 的信号。status_code 需为整数且在 100-599。potential_findings 仅保留 SQL Injection / SSRF，最多检查前 10 个候选。

为何不用字符串截断代替

把整个 observation.data 转字符串，即使截断也会保留源码中的指令、秘密或任意嵌套内容。按字段和类型白名单提取更容易审查实际暴露了什么。

v0.7 的实际限制

搜索结果和源码不会传给 LLM Planner；嵌套 http_baseline 的状态也不进入当前顶层 status_code 白名单。项目尚未实现工具感知规划和源码语义回传，不能介绍成模型已阅读完整源码后自主验证漏洞。

自测

工具 data 中添加 instruction="ignore policy"，Planner 会收到吗？当前构建器不会复制这个字段。用户 goal 本身仍是不可信输入，最终权限依赖执行门控。

函数定位（项目内相对路径与当前行号）

runtime/planning_context.py : 11-26 PlanningContextBuilder.build

runtime/planning_context.py : 39-63 PlanningContextBuilder._observations

10 DemoPlanner 与 LLMPlanner

两个 Planner 使用同一接口，因此离线可复现的演示与真实模型适配共用 Runtime。

DemoPlanner.plan_phase()

ANALYZE 生成 query="execute("、max_matches=20 的分析任务。VERIFY 只为现有 POTENTIAL/UNCONFIRMED 的 SQL Injection 或 SSRF 生成验证任务。REPORT 生成 Mock 报告。默认真实搜索不生成 Finding，所以 VERIFY 通常为空。

LLMPlanner.__init__()

要求非空模型名，可以注入 client 做离线测试。默认创建 OpenAI client，timeout=60.0、max_retries=2。初始化本身不发送模型请求。

LLMPlanner.plan_phase()

使用 responses.parse，text_format=PlanProposal，max_output_tokens=4096、store=False。系统提示说明 Runtime 已决定阶段、任务上限和目标规则；用户消息携带 goal 和 JSON context。

异常处理

响应不是 completed 时返回 INCOMPLETE_PLAN_RESPONSE；没有 output_parsed 时返回 NO_PARSED_PLAN。请求、解析等异常包装为 PLANNER_REQUEST_FAILED，避免把原始 API 响应或敏感信息写到运行状态。之后仍需 PlanValidator 的语义校验。

自测与测试

tests/test_planning.py 中的 Mock 与离线 transport 验证参数和错误处理。integration_tests/ 真实调用需显式开启，会产生 API 费用。本讲义描述当前适配代码，没有实测真实服务成功率。

函数定位（项目内相对路径与当前行号）

planner/demo_planner.py : 7-29 DemoPlanner.plan_phase

planner/llm_planner.py : 11-20 LLMPlanner.__init__

planner/llm_planner.py : 22-62 LLMPlanner.plan_phase

11 依赖图校验

TaskScheduler.validate_graph(state) 验证 Runtime 层面的完整图，区别于 PlanValidator 对单次提案局部 key 的检查。

基础检查

先构建 task_map。每个任务的 phase 必须在 ANALYZE、VERIFY、REPORT 中，ID 不能重复。然后检查每条依赖的 ID 必须存在，不允许任务依赖自身。

跨阶段依赖

为阶段赋顺序 0、1、2。若依赖任务的阶段大于当前任务阶段，拒绝 FUTURE_PHASE_DEPENDENCY，防止 ANALYZE 等待 VERIFY 导致阶段死锁。允许 Runtime 图引用更早阶段的任务，但 LLM 单次提案只允许内部依赖。

环检测

DFS 访问时把当前节点放到 visiting，递归其依赖；再次遇到 visiting 表示环。处理完成后移入 visited。两个集合分别代表当前递归路径与已经完成检查的节点。

复杂度与限制

图校验大体为 O(V+E)。当前每阶段最多 10 个 LLM 任务且只有三个阶段，图规模很小。不能据此声称支持大规模 DAG 或分布式调度。

自测

A 在 VERIFY，依赖 REPORT 的 B，即使图无环是否允许？不允许，因为未来阶段依赖违反阶段推进顺序。对应测试见 tests/test_scheduler.py、tests/test_planning.py。

函数定位（项目内相对路径与当前行号）

runtime/scheduler.py : 7-79 TaskScheduler.validate_graph

12 就绪刷新与失败传播

TaskScheduler.refresh(state) 负责状态变化，ready_tasks(state) 负责选择，Supervisor 负责真正执行。

refresh()

只处理当前阶段的 PENDING 任务。任一依赖 FAILED、BLOCKED 或 TIMEOUT 时，把任务标记 BLOCKED，block_reason=DEPENDENCY_FAILED。全部依赖 COMPLETED 时，改为 READY。无依赖列表的 all([]) 为 True，因此可立即就绪。

为什么有 while changed

A 失败导致 B 阻断，B 又导致 C 阻断。循环直到状态不再改变，让失败能跨多级依赖传播，而不依赖任务列表的排列顺序。

ready_tasks()

返回当前阶段且状态为 READY 的任务。AgentRuntime 按列表逐个派发，没有线程池或 asyncio 并发。Scheduler 的 READY 是资格状态，不是已经取得执行权限。

PhaseManager

can_advance 要求阶段已规划，且该阶段所有任务均为 COMPLETED、FAILED、BLOCKED 或 TIMEOUT。WAITING_CONFIRMATION 不是终态。next_phase 依序进入 VERIFY、REPORT、END。

容易误解的结果

工作流 COMPLETED 仍可有失败任务，因此要读 failed_task_ids。RuntimeRunner 是 v0.5 旧循环，执行已有任务，不负责 LLM 规划。自测：等待批准的依赖会让下游 READY 吗？不会，只有 COMPLETED 满足依赖。

函数定位（项目内相对路径与当前行号）

runtime/scheduler.py : 81-131 TaskScheduler.refresh

runtime/scheduler.py : 133-139 TaskScheduler.ready_tasks

runtime/phase_manager.py : 5-11 PhaseManager.can_advance

runtime/runner.py : 19-61 RuntimeRunner.run_until_blocked

13 Supervisor.delegate 执行入口

delegate(task, state) 将一个 Runtime Task 转为安全检查、待审批动作或 Agent 执行。

请求与状态

RequestValidator.validate 使用 TaskRequest.model_validate 从属性读取字段。无效 schema 通过 _deny_task 记录 BLOCKED。重复派发已完成、执行中或等待中的任务会返回 INVALID_TASK_STATE，不破坏原状态。

安全与所有权

_check_security 检查阶段、Agent、capability、Policy 和 Scope。然后 _task_error 确认 State 内仅有一个同 ID 任务且对象就是传入 task，并检查依赖已完成；finding_error 检查 Finding 关联。

分流

DENY 分支记录拒绝原因。CONFIRM 分支调用 ApprovalStore.create，保存快照并设 WAITING_CONFIRMATION。ALLOW 分支同样创建 ApprovalSnapshot.from_task 后交 _execute，避免直接把可变 Task 交给 Agent。

错误类别

无效关联 Finding 的特定分支可能将任务置 FAILED；策略或依赖拒绝通常置 BLOCKED。返回值的 DENIED 是一次调用结果，Task.status 的 BLOCKED 是运行状态，两者不必使用同一字符串。

自测

构造一个与 State 中任务字段完全相同的新 Task 对象，能直接执行吗？不能，_task_error 使用对象身份匹配，阻止非 Runtime 持有的替身任务。

函数定位（项目内相对路径与当前行号）

runtime/supervisor.py : 78-160 Supervisor.delegate

runtime/supervisor.py : 393-401 Supervisor._task_error

runtime/validator.py : 7-11 RequestValidator.validate

14 Workflow、Policy、Scope 与 Capability

这四类检查解决不同问题。面试时最好分别用一个失败例子解释，而不是合称“权限检查”。

Workflow 和 phase

WorkflowEngine.is_allowed 判断当前阶段允许的 action。例如 generate_report 只能在 REPORT；还要 request.phase 与 state.phase 相等，防止一个未来阶段任务借用同名 action 提前执行。

Capability

AgentRegistry.get 找到真实实例及 capability。任务 action 必须出现在指定 Agent 的能力列表中。CodeAnalysisAgent 不能因为任务写了 verify_ssrf 就变成验证 Agent。

Policy

PolicyEngine.evaluate 对未知 action 默认 DENY。默认 verify_ssrf=CONFIRM、execute_command=DENY，其余列出的分析、读取、baseline、SQLi 模拟和报告为 ALLOW。Workflow 中存在 execute_command 不代表能执行，还会被 capability 或 Policy 拒绝。

Scope

ScopeEngine.is_allowed 使用请求 target 与 state.target 字符串精确相等。不做子域名、子目录、URL 等价形式的推断。真实文件的路径范围还要在工具层检查，不能把 target 精确匹配当作文件访问检查。

HITL 的作用

approval_granted=True 只解除 CONFIRM 这一条件，不能把 DENY 变成 ALLOW。自测：批准后策略变为 DENY，是否继续？Supervisor.approve 会重查并拒绝。

函数定位（项目内相对路径与当前行号）

runtime/supervisor.py : 296-391 Supervisor._check_security

runtime/workflow.py : 22-29 WorkflowEngine.is_allowed

runtime/policy.py : 24-28 PolicyEngine.evaluate

runtime/scope.py : 3-9 ScopeEngine.is_allowed

runtime/registry.py : 5-9 AgentRegistry.register

15 审批快照与状态存储

ApprovalSnapshot 和 ApprovalStore 把“用户批准哪一个动作”固定下来，避免批准的是旧内容而执行的是新内容。

ApprovalSnapshot.from_task()

捕获 id、agent、action、target、finding_id、phase、dependencies 和 arguments。json.dumps 使用排序 key、紧凑分隔符、allow_nan=False，保存为 arguments_json。依赖列表转为 tuple，Pydantic frozen=True 阻止字段重新赋值。

arguments 属性

json.loads 每次返回新的字典。因此 Agent 修改 snapshot.arguments 得到的嵌套对象不会修改原 Task，也不会修改快照存储字符串。单靠 frozen=True 冻结一个含可变 dict 的模型不能达到这一效果。

ApprovalStore.create()

分配 A001 形式的 action_id，将 PendingAction 存入 actions，并在 _owners 存创建它的 State 对象。belongs_to 使用 is 比较对象身份，而不是 State 内字段相等。

生命周期

WAITING_CONFIRMATION 经 approve 变为 APPROVED，执行后 mark_executed 变为 EXECUTED，拒绝时 mark_rejected 变为 REJECTED。后续不能再批准同一个已消费动作。所有记录在内存中。

自测

快照是签名或加密凭据吗？不是。当前是进程内不可变数据和身份绑定机制，未包含审批人的身份认证、过期时间、持久签名或跨进程恢复。

函数定位（项目内相对路径与当前行号）

models/approval.py : 37-56 ApprovalSnapshot.from_task

models/approval.py : 59-62 ApprovalSnapshot.arguments

runtime/approval.py : 17-35 ApprovalStore.create

runtime/approval.py : 37-38 ApprovalStore.belongs_to

16 Supervisor.approve 批准后复检

审批是一次延迟执行。延迟期间环境和任务可能发生变化，所以批准时必须重新验证。

先验证待批动作

未知 action_id 拒绝；不属于当前 State 拒绝；PendingAction 不在 WAITING_CONFIRMATION 拒绝。找到对应 Runtime Task 后，也要求其仍处于 WAITING_CONFIRMATION。

_matches_snapshot()

逐项比较 id、agent、action、target、arguments、finding_id、phase 和 dependencies。任一变化都拒绝 APPROVED_REQUEST_MISMATCH，防止任务参数漂移或审批后偷偷新增依赖。

再次检查执行条件

从快照构造请求，调用 _check_security(..., approval_granted=True)，重查当前 Policy、Scope、phase、capability。再检查 Runtime 任务归属、依赖完成和 Finding 关联，然后执行冻结快照。

审批消费与恢复

_execute 返回后将 PendingAction 标记 EXECUTED。即使 Agent 执行失败，也不重放同一次审批。调用方保留同一 runtime、state，再调用 run_until_blocked 继续后续任务。CLI 本身不提供交互批准入口。

测试定位

tests/test_security_regressions.py 验证参数篡改、原 State 绑定、phase/依赖变化与重复派发；tests/test_runtime.py 验证审批链。自测：批准等于无限期获得这个 action 的权限吗？不是，它只针对这份待执行快照。

函数定位（项目内相对路径与当前行号）

runtime/supervisor.py : 163-282 Supervisor.approve

runtime/supervisor.py : 566-581 Supervisor._matches_snapshot

17 Supervisor._execute 与结果校验

执行入口检查通过之后，仍需要验证 Agent 返回的是当前任务的结果。

准备与调用

查找关联 Finding，把任务置 RUNNING。调用 _build_tool_context，从 State.target 与 State.authorized_origin 构造 ToolContext，然后执行 agent.run(execution_task, tool_context)。Agent 不接收全局 State。

异常与 Schema

Agent 抛异常时将任务设 FAILED。返回值由 ObservationValidator 校验，execution_status 只允许 SUCCESS、FAILED、TIMEOUT。数据形状不合法得到 INVALID_OBSERVATION。

结果身份绑定

observation.agent 必须与 execution_task.agent 相等，task_id 也必须相等。否则记 OBSERVATION_MISMATCH 并失败，避免把其他任务结果混入当前执行链。通过后才保存 Observation。

更新顺序

非 SUCCESS 观察直接映射为任务状态并返回。成功观察先记录允许的潜在 Finding，再提取 Evidence 并保存；已有 Finding 则经 Evaluator 计算新状态，最后把 Task 设 COMPLETED。

自测

工具返回 SUCCESS，但 Observation.task_id 写错怎么办？任务失败，错误 Observation 不入库。代码使用统一的 run(task, context) 接口，没有捕获 TypeError 后再次调用旧接口的回退，避免重复副作用。

函数定位（项目内相对路径与当前行号）

runtime/supervisor.py : 404-411 Supervisor._build_tool_context

runtime/supervisor.py : 413-563 Supervisor._execute

runtime/validator.py : 16-20 ObservationValidator.validate

18 StateManager 与 Finding 关联

StateManager 集中提供状态变更方法，但当前 State 是普通进程内对象，不能将其理解为不可绕过的数据库权限层。

常规操作

add_task、add_observation、add_evidence、add_finding 向列表追加。find_task、find_finding 按 ID 查找。update_task_status 修改任务状态；它自身并未实现完整 Task 状态迁移表，调用顺序主要由 Runtime 和 Supervisor 控制。

Finding 状态迁移

POTENTIAL 可以进入 CONFIRMED、REJECTED 或 UNCONFIRMED；UNCONFIRMED 可以进入 CONFIRMED 或 REJECTED；CONFIRMED 和 REJECTED 没有后续迁移。同状态更新不做事，非法迁移抛 INVALID_FINDING_TRANSITION。

record_potential_findings()

仅接收 analyze_code 的 SUCCESS，读取 potential_findings 列表前 10 项，只接受 SQL Injection/SSRF。按类型和目标去重，由 Runtime 分配 F1 等 ID，使用任务目标与默认 POTENTIAL。当前真实 CodeAnalysisAgent 不提供此字段，保留该入口用于扩展及测试。

finding_error()

要求引用存在且唯一，Finding、Task、State 目标一致；action 和 Finding 类型匹配，状态未终结。规划时要求验证动作必须引用 Finding，旧手动派发接口允许没有 finding_id，但不会提取关联证据。

自测

把 SQL Injection 的 Finding 传给 verify_ssrf，会得到 FINDING_TYPE_MISMATCH。这防止一个动作的输出错误确认另一个类型或另一个目标的漏洞。

函数定位（项目内相对路径与当前行号）

runtime/state_manager.py : 75-111 StateManager.update_finding_status

runtime/state_manager.py : 157-177 StateManager.record_potential_findings

runtime/finding_rules.py : 7-27 finding_error

19 EvidenceExtractor 与 FindingEvaluator

证据链的重点是可追溯关联和状态语义。当前规则简单，必须说明教学范围。

extract() 的入口条件

Observation 必须 SUCCESS，Task 必须有 finding_id。只有 verify_sqli 和 verify_ssrf 进入相应提取函数。read_file、analyze_code、check_http_baseline 不生成 Evidence。

SQLi 规则

response 中出现 "SQL syntax" 生成 SQL_ERROR_SIGNAL，强度 MEDIUM。boolean_difference 严格为 True 时生成 BOOLEAN_RESPONSE_DIFFERENCE，强度 STRONG。单凭 500 状态码不匹配这些规则。

SSRF 规则

callback_received=True 生成 STRONG 的 OUT_OF_BAND_CALLBACK；否则 request_completed=True 生成 WEAK 的 REQUEST_COMPLETED。Evidence ID 由提取器按 E001 分配，并绑定 task_id 和 finding_id。

evaluate()

筛选与当前 Finding.id 对应的证据，只要有 STRONG 就返回 CONFIRMED，否则返回 UNCONFIRMED。没有证据也不自动 REJECTED。实际内置 verify_sqli 只返回错误字符串，verify_ssrf 返回 request_completed=True、callback_received=False，因此仅能提供弱或中等模拟信号。

自测

如何防止把模拟证据混入生产？当前模型没有强制的模拟来源标记。这是后续应补齐的 provenance 与隔离规则，不能仅因为有 Evidence 类就声称已具备生产级证据可信度。

函数定位（项目内相对路径与当前行号）

runtime/evidence.py : 11-37 EvidenceExtractor.extract

runtime/evidence.py : 50-97 EvidenceExtractor._extract_sqli

runtime/evidence.py : 99-141 EvidenceExtractor._extract_ssrf

runtime/evaluator.py : 3-27 FindingEvaluator.evaluate

20 工具契约与注册表

ToolRegistry 验证“工具长什么样”，ToolExecutor 决定“这一次能否调用”，ToolContext 提供“授权在哪里”。

ToolContext

冻结 Pydantic 模型，禁止额外字段，要求 task_id、agent、action，授权根目录和 Origin 可为空。工具缺少对应授权值时拒绝执行。上下文由 Supervisor 构造，业务 arguments 不能覆盖它。

ToolResult

只允许 SUCCESS 或 ERROR，含 tool_name、data 和 error。没有 TIMEOUT 枚举；当前工具超时进入 ERROR，再由 Agent 映射为 FAILED Observation。

ToolRegistry.register()

要求 name 是非空合法标识符且未重复，args_model 必须是 BaseModel 子类，execute 必须可调用。通过后放入 _tools。get 按名称获取，names 返回 tuple。

与 AgentRegistry 的差异

AgentRegistry 注册实例与 capability，注册同名目前可覆盖。ToolRegistry 对重复工具名明确报 DUPLICATE_TOOL。两套 Registry 的职责和校验力度不同，不应笼统说注册表均防覆盖。

自测

有一个带 execute 方法的普通对象就能注册吗？还需要合法名称和 Pydantic 参数模型。相关测试：test_tool_models.py、test_tool_registry.py、test_tools_contract.py。

函数定位（项目内相对路径与当前行号）

models/tool.py : 26-41 ToolContext

models/tool.py : 8-23 ToolResult

runtime/tool_registry.py : 11-38 ToolRegistry.register

tools/base.py : 8-19 Tool

21 ToolExecutor.execute 工具权限

工具执行的安全边界由 (agent, action) 的白名单和每个工具的参数模型共同形成。

第一层：调用身份

必须是 ToolContext，tool_name 必须合法且在当前 (context.agent, context.action) 的允许集中。未知组合使用空集合。权限拒绝抛 ToolExecutionDenied，查不到工具也拒绝。

第二层：原始参数校验

arguments 必须是 dict，而不能直接传入已构造模型。调用 tool.args_model.model_validate(arguments)，验证失败抛 ToolArgumentsInvalid，避免用预构造模型绕过验证。

第三层：实际执行

tool.execute(parsed_arguments, context) 抛出的异常转换为 ToolResult(ERROR)，只保存异常类型名。正常返回必须是 dict，并再次构造成 ToolResult，否则返回 INVALID_TOOL_RESULT。

错误传播

权限或参数错误是抛异常，工具运行错误是返回 ERROR。CodeAnalysisAgent 与 VerificationAgent 处理这两类路径，统一输出 FAILED Observation，Supervisor 再把 Task 标为 FAILED。这和执行前策略拒绝导致 BLOCKED 不同。

自测

Agent capability 包含 read_file，能否调用 HTTP 工具？不能，工具白名单独立检查。ToolContext 的 frozen 并不是对任意 Python 插件的隔离，恶意插件仍可能绕开约定直接 I/O，需要操作系统级限制。

函数定位（项目内相对路径与当前行号）

runtime/tool_executor.py : 60-149 ToolExecutor.execute

agents/code_analysis.py : 33-48 CodeAnalysisAgent._execute_tool

22 CodeAnalysisAgent 与报告 Agent

Agent 的职责是解释动作、调用允许工具、返回观察，不自行修改 Runtime 的全局状态。

run() 分流

analyze_code 调 _analyze_code，read_file 调 _read_file，其他动作返回 FAILED 和 UNSUPPORTED_ACTION。输入是 ApprovalSnapshot 与 ToolContext。

_analyze_code()

读取快照 arguments，query 必须是字符串。提交 search_code 参数 query 和 max_matches，Agent 的默认值为 20。SearchCodeArgs 本身默认 30，但这里总会传入 Agent 选定的值。

_read_file() 与 _execute_tool()

path 必须是字符串，调用 read_source。_execute_tool 捕获预期权限/参数错误，将工具 ERROR 映射为 FAILED；成功时把 tool_name 和结果字段放进 Observation.data。不会再添加硬编码 SQLi 候选。

ReportAgent.run()

只返回 demo_only=True 与 Mock report 完成消息。CLI 通过 State 中的 findings/evidence 单独打印结果。不能把这个 Agent 描述成自动撰写完整漏洞报告的能力。

自测

search_code 命中 cursor.execute(query) 就代表 SQL 注入吗？还缺用户输入可控性、传播路径、危险调用语义和有效验证，因此只记录命中。相关测试：tests/test_agent_tools.py。

函数定位（项目内相对路径与当前行号）

agents/code_analysis.py : 10-15 CodeAnalysisAgent.run

agents/code_analysis.py : 17-25 CodeAnalysisAgent._analyze_code

agents/code_analysis.py : 27-31 CodeAnalysisAgent._read_file

agents/report.py : 4-13 ReportAgent.run

23 文件路径校验与有界读取

tools/file_policy.py 提供共享的文件安全规则，ReadSourceTool 和 SearchCodeTool 分别调用所需部分。

get_authorized_root()

从可信 ToolContext 取 authorized_root，expanduser 后 resolve(strict=True)，要求目录存在且确实是目录。路径范围来自 Runtime，而不是搜索或读取的业务参数。

resolve_source_path()

只接受 / 分隔的相对路径，拒绝 NUL、反斜杠、绝对路径、Windows drive，以及空段、.、..。拼接后再 resolve(strict=True)，通过 relative_to(root) 检查真实路径，阻止指向授权根外的软链接。

文件类型限制

要求普通文件，拒绝配置的敏感文件名，例如 .env、id_rsa，扩展名需在源码白名单内。这是明确的规则集合，不是自动识别所有秘密。读取任意 .json 不等于其中没有凭据。

read_bounded_bytes()

以二进制打开后用 fstat 检查普通文件及大小，再读 max_bytes+1；超过上限时报 FILE_TOO_LARGE。双重检查比只看文件名或初始 stat 更可靠。单文件上限 256 KiB。

适用边界

路径检查和打开仍有时间间隔，无法声称完全抵抗恶意进程并发替换路径的竞态。ReadSourceTool 也没有直接复用搜索时的忽略目录规则。自测：授权根内的链接指向根外，会在 relative_to 检查被拒绝。

函数定位（项目内相对路径与当前行号）

tools/file_policy.py : 52-74 get_authorized_root

tools/file_policy.py : 77-148 resolve_source_path

tools/file_policy.py : 151-178 read_bounded_bytes

24 ReadSourceTool.execute

把相对文件路径转换成有范围、有大小限制的 UTF-8 源码读取。

输入

ReadSourceArgs 要求 path 非空且不超过 240 字符，extra="forbid" 拒绝额外授权根字段。执行器先校验参数，再调用工具。

调用链

get_authorized_root(context) 获取可信根目录。resolve_source_path(root, args.path) 做路径及类型限制。read_bounded_bytes(source_path, MAX_SOURCE_FILE_BYTES) 限制实际读取。最后按 UTF-8 解码，返回 relative_path、content 与 bytes。

失败如何返回

缺少授权根、路径越界、文件不存在、文件超限或非 UTF-8 都可能抛异常。ToolExecutor 将运行异常封装为 ERROR，仅保留类型名；Agent 记录 TOOL_EXECUTION_FAILED，任务最终 FAILED。

示例与测试

在授权临时目录中创建 sample.py，读取 path="sample.py" 应得到真实内容。将路径改为 ../secret.py、绝对路径或指向根外的软链接应拒绝。tests/test_read_source.py 验证这些边界，test_agent_tools.py 验证经 Agent 的链路。

自测

返回 content 后是否会自动传给 LLM？不会，Observation.data 可保存源码，但 PlanningContextBuilder 的白名单不会把 content 复制进去。

源码节选（省略空行，仅展示函数开头）

```
31      def execute(
32          self,
33          args: ReadSourceArgs,
34          context: ToolContext,
35      ) -> dict:
37          root = get_authorized_root(
38              context
39          )
41          source_path = resolve_source_path(
42              root,
43              args.path,
44          )
46          content_bytes = read_bounded_bytes(
47              source_path,
48              MAX_SOURCE_FILE_BYTES,
```

函数定位（项目内相对路径与当前行号）

tools/read_source.py : 13-22 ReadSourceArgs

tools/read_source.py : 31-67 ReadSourceTool.execute

25 SearchCodeTool.execute

当前分析能力是受限的字面搜索。它提供路径、行号和片段，供后续分析使用。

输入约束

query 长度 1-100，不能为空白或含 NUL。max_matches 为 1-80。args_model 禁额外字段。搜索对 query 与每行使用 casefold，所以不区分大小写，但不支持任意正则。

遍历与过滤

iter_project_files 使用 os.scandir 的条目预算，跳过 .git、.venv、node_modules 等目录，也跳过目录软链接和 Windows junction。候选文件 resolve 后检查仍在根内，过滤敏感文件名、扩展名和非普通文件。

资源预算

最多 2000 个目录条目、400 次文件读取尝试、单文件 256 KiB、总计 16 MiB、匹配最多 80 条，每条 snippet 最多 240 字符。实际结果达到限制或遇到某些读取/解码问题会标 truncated。

输出与含义

返回 query、matches、files_scanned、bytes_scanned、truncated。matches 每项有相对 path、从 1 开始的 line、snippet。truncated=True 表示结果可能不完整，不代表目标有漏洞。

自测

files_scanned=0、matches=[] 能证明没有风险吗？不能，可能文件格式、大小或预算过滤使搜索不完整，且字面检索本身不覆盖所有等价调用。测试见 test_search_code.py。

函数定位（项目内相对路径与当前行号）

tools/search_code.py : 29-55 SearchCodeArgs

tools/search_code.py : 64-202 SearchCodeTool.execute

tools/file_policy.py : 187-244 iter_project_files

26 HTTP 参数与 Origin 限制

ControlledHttpArgs 限制路径，_get_authorized_origin 限制可信网络目标，两部分合起来构造请求 URL。

路径规则

path 默认 /，必须单斜杠开头，不能以 // 开头。拒绝反斜杠、查询串 ?、片段 # 和控制字符。逐次 URL 解码最多 10 轮，进一步拒绝编码后的控制符、协议形式和 . / .. 段。method 仅 GET 或 HEAD。

Origin 规则

从 ToolContext.authorized_origin 读取，拒绝空值。urlsplit 前先拒绝空白/控制字符，再要求 http/https、合法 hostname 和 port，无用户信息、查询、片段或非根路径。不能在参数里传另一个主机覆盖它。

构造方式

执行时使用 origin + args.path，不使用可被绝对 URL 改写目标的通用拼接行为。禁止跳转和代理，降低请求意外转向授权外目标的风险。

适用边界

这是应用层固定 Origin 约束，不是完整 SSRF 防护系统。没有实现 DNS 固定解析、IP 地址段策略或 DNS 重绑定防护。允许的 Origin 应由调用方明确授权，GET/HEAD 也不能保证目标应用没有错误设计的副作用。

自测

/a?x=1 在当前版本允许吗？不允许。//evil.invalid/ 或 /%2e%2e/secret 也会在执行前拒绝。相关用例见 tests/test_controlled_http.py。

函数定位（项目内相对路径与当前行号）

tools/controlled_http.py : 46-108 ControlledHttpArgs.validate_path

tools/controlled_http.py : 132-186 ControlledHttpCheckTool._get_authorized_origin

27 真实 HTTP 请求与 baseline 观察

ControlledHttpCheckTool 产生响应元信息；VerificationAgent._check_http 将其保存为独立的基线数据。

请求执行

build_opener(ProxyHandler({}), NoRedirectHandler()) 禁用代理与自动跳转。Request 采用限制的方法和固定 User-Agent；opener.open 使用 3 秒请求超时。Agent 默认 HEAD，直接工具参数模型默认 GET，两个默认值需要区分。

读取上限的精确含义

GET 调用 response.read(64*1024+1)，多读 1 字节判断是否截断。返回 body_bytes_read 最大为 65536，truncated 表示是否超过。HEAD 不读 body，工具不返回响应正文。不要表述成网络实际最多读 65536 字节。

HTTPError 与网络异常

urllib 把部分 HTTP 状态变为 HTTPError；工具捕获后返回状态码元信息，3xx 标 redirect_not_followed=True，并关闭响应。连接、超时等异常交给 ToolExecutor 变为 ERROR。HTTP 500 可以是工具执行 SUCCESS，因为成功观察到了响应。

观察格式

VerificationAgent._check_http 成功返回 data={tool_name, http_baseline: result.data}。它不生成 Finding/Evidence。工具错误或参数拒绝映射 FAILED；网络超时在当前工具链同样是 FAILED，而不是 TIMEOUT。

自测

3 秒是整个任务绝对时间上限吗？不是，此处传给 urllib 的 timeout 不等于 Runtime 实施了完整任务 deadline。真实拒绝路径用服务器请求记录来验证。

函数定位（项目内相对路径与当前行号）

tools/controlled_http.py : 113-123 NoRedirectHandler.redirect_request

tools/controlled_http.py : 189-265 ControlledHttpCheckTool.execute

agents/verification.py : 32-54 VerificationAgent._check_http

28 端到端源码搜索推演

用实际离线命令把所有组件串起来：.venv/Scripts/python.exe main.py --target ./tools。

1. 规划与编译

main 创建 target="./tools" 的 State，初始 phase=ANALYZE。DemoPlanner 提议 analyze_code，参数 query="execute("。Validator 接受合法 Agent 和同一目标；Compiler 分配 T1。

2. 执行与观察

Scheduler 将无依赖 T1 置 READY。Supervisor 检查后生成冻结快照与 authorized_root="./tools" 的 ToolContext。CodeAnalysisAgent 经 ToolExecutor 调用 SearchCodeTool。返回真实命中后 Observation 绑定 T1，T1 变 COMPLETED。

3. 后续阶段

真实分析未提供 potential_findings，Finding 列表为空。VERIFY 因此是空计划，标记已规划后推进。REPORT 创建 T2，由 ReportAgent 返回演示观察，随后进入 END。

本次实测输出

扫描 6 个文件，命中 base.py、controlled_http.py、read_source.py、search_code.py 中的 execute(，共 4 条。结果 COMPLETED，failed_task_ids=[]，Evidence=[]，Findings=[]。这些只是当前源码快照的运行值。

自测

如果目标目录不存在，分析任务会 FAILED，但工作流仍可能到 END。需要同时看 failed_task_ids 与 CLI 退出码，不能只截取 COMPLETED 作为成功证据。

函数定位（项目内相对路径与当前行号）

main.py : 63-85 main

planner/demo_planner.py : 7-29 DemoPlanner.plan_phase

runtime/orchestrator.py : 23-81 AgentRuntime.run_until_blocked

29 安全测试的阅读方法

先问测试要保护什么性质，再看输入、断言和真实副作用。测试数量本身不能代表覆盖率。

审批与任务归属

test_security_regressions.py 检查重复派发、参数深层修改、跨 State 审批、阶段与依赖变化、冻结请求、跨目标/类型 Finding。重点看拒绝后原任务和 Agent 调用次数是否符合预期。

规划与图

test_planning.py 验证结构化提案、空阶段、规划失败不提交、上下文白名单和恢复时不重复规划。test_scheduler.py 检查环、缺失依赖、失败/超时传播等。

工具与接线

test_tool_executor.py 关注权限与参数契约。test_read_source.py、test_search_code.py、test_controlled_http.py 关注 I/O 边界。test_runtime_tool_context.py 检查授权来自 State 以及 baseline 不污染证据。

本地真实 HTTP

test_http_loopback_integration.py 启动 127.0.0.1 临时服务。GET/HEAD 验证真实请求；302 验证不请求 Location；大响应验证截断；非法路径后 server.received 不增加，证明拒绝路径没有实际发出请求。

自测与结论

“mock 返回 200”只能验证处理逻辑；真实回环请求可以补充网络行为证据。当前 181 项通过，不能延伸为公网环境可靠性、性能基准、全分支覆盖或所有漏洞防护。

30 复习实验与运行命令

下面实验均使用现有离线用例或授权源码目录，不需要 API Key。推荐从命令输出定位对应函数。

实验一：完整演示

在项目根运行 .venv/Scripts/python.exe main.py --target ./tools。解释两个 Task、空 VERIFY、两条 Observation 和空 Finding/Evidence。再用 --target ./does-not-exist 对照失败任务与工作流状态。

实验二：审批与篡改

运行 .venv/Scripts/python.exe -m unittest tests.test_security_regressions -v。重点阅读 test_approval_snapshot_freezes_arguments_and_rejects_mutation，解释 deepcopy、JSON 快照和执行时复检各解决哪一个对象共享问题。

实验三：真实回环请求

运行 .venv/Scripts/python.exe -m unittest tests.test_http_loopback_integration -v。给 GET、HEAD、redirect 和非法路径分别写下预期 server.received，再对照断言。

实验四：规划与工具

运行 .venv/Scripts/python.exe -m unittest tests.test_planning tests.test_tool_executor -v。手动列出 UNKNOWN_AGENT、CYCLIC_PLAN、TOOL_NOT_ALLOWED_FOR_ACTION 三种错误分别发生在哪一层。

实验五：扩展设计

仅做纸面设计：新增一个受限工具时，需要参数模型、execute 契约、Registry 注册、(agent, action) 权限、Agent 分流、必要 Workflow/Policy/Action 定义及测试。先列安全边界再编写实现。完整回归命令为 .venv/Scripts/python.exe -m unittest discover -s tests -q。

31 面试高频问题与回答要点

用项目里可定位的函数回答，比抽象说“做了多层安全”更有说服力。

为什么不用 LLM 直接调用工具？

模型计划是建议。权限由 Runtime 根据 State、Policy、Scope、capability、审批和工具白名单决定。对应 PlanValidator.validate、Supervisor._check_security、ToolExecutor.execute。

审批后为什么还要复检？

审批和执行之间任务或环境可能变化。快照比较保证批准内容一致，重新检查保证当前仍然允许执行。只验证其中一个不能覆盖另一类变化。

如何处理幻觉和提示注入？

结构化输出限制形状，语义校验拒绝不存在的 Agent、错误目标和依赖，上下文白名单减少不可信原文暴露。最终执行权限独立于模型。不能声称完全解决提示注入。

为什么任务成功却没有 Finding？

任务状态描述工作完成，Finding 描述漏洞判断。源码命中和 HTTP 元信息不足以确认漏洞。v0.7 的真实工具不自动生成漏洞证据，模拟验证规则也不能当作生产结论。

最有价值的测试是什么？

可展示非法 HTTP 路径没有产生请求、重定向未跟随、审批篡改后 Agent 没有执行。它们直接检验安全属性，而不是只检查返回字符串。

32 局限、改进与个人贡献表达

材料用于帮助你理解和陈述真实完成的工作，不替代对自己贡献范围的说明。

能力局限

LLM 不接收完整源码；搜索只是字面匹配。SQLi/SSRF 与报告仍是模拟。证据强度规则简单，缺少强制的来源标记。没有持久化、并发执行、Observation 级重规划或任务全局 deadline。

边界局限

frozen 对象与白名单属于应用层控制，不是任意 Python 插件沙箱。文件访问未完全处理并发路径替换竞态；固定 Origin 未实现 DNS 重绑定防护。StateManager 也不是数据库事务和授权层。

合理的后续顺序

先增加有来源标记的真实验证输出和报告，再设计持久化与恢复协议。引入并发时补充锁、幂等和审批一致性。支持第三方插件时，增加独立进程、最小文件/网络权限与资源配额。以上均为建议，尚未实现。

个人贡献怎么说

分别列出自己设计的接口、亲自修改的函数、定位过的缺陷、编写或验证的测试。如使用 AI 辅助，应如实说明辅助范围，并能现场解释关键代码和失败分支。不要用“独立开发全部”替代事实。

90 秒介绍模板

我做了一个安全任务 Agent Runtime。模型只提出计划，Runtime 负责阶段、权限、依赖和审批，再通过受控工具读取源码和做 HTTP 基线检查。我重点研究执行时复检、冻结审批快照和结果证据分离。项目经过七轮迭代，当前 181 项离线测试通过。真实漏洞验证和报告仍是教学实现，下一步会完善证据来源与真实验证插件。